

TSUMUGIタイムスタンプ検証手順書

1.0版
2026年10月10日
TSUMUGI Labo.

改訂履歴

版	変更日付	変更箇所	変更内容	発行責任者
0.1	2026年9月28日	—	草案作成(公開前。 公開に際し運用規 程とともに1.0とす る)	TSUMUGI Labo. 渋谷 憲
1.0	2026年10月10日	—	公開に伴い運用規 程とともに1.0とす る	TSUMUGI Labo. 渋谷 憲

目次

1.はじめに	5
1.1.本書の目的	5
1.2.前提とする環境	5
1.3.用意するもの	5
1.4.運用規程4.1.4との対応	5
2.準備	6
2.1.ルート認証局証明書の取得と確認	6
2.2. CRLの取得	6
3.タイムスタンプトークンの取り出し	6
3.1.分離トークン	6
3.2. PDF	7
4.トークンの署名・証明書・失効の検証	7
4.1.トークンの内容の表示	7
4.2.署名・証明書・失効の検証	8
4.3. TSA証明書の有効期間が過ぎている場合	8
5.存在証明の取り出し(第一項)	8
5.1.属性の位置	8
5.2.存在証明の構造	8
5.3.取り出し	9
6.存在証明の検証(第四項～第六項)	9
6.1. payloadの構造	9
6.2.対象データとの結びつき	10
6.3.包含証明の再計算(第四項)	10
6.4. checkpointの構造	10
6.5.ログの署名の検証(第五項)	11
6.6. witnessの連署の検証(第五項)	11
6.7.時刻の照合(第六項)	12
7. TSA証明書が失効している場合	13
7.1.基準日時	13
7.2.判定	13

8.判定のまとめ 13

1.はじめに

1.1.本書の目的

本書は、TSUMUGIタイムスタンプサービス運用規程(以下「運用規程」といいます。)4.1.4に定めるタイムスタンプトークンの検証を、TSUMUGI Labo.(以下「当方」といいます。)のソフトウェアを用いずに行う手順を示します。

本書の手順のうち、トークンの署名・証明書・失効の検証、及び公開透明性ログとwitnessの署名の検証は、OpenSSLのコマンドで行えます。PDFからのトークンの取り出し、JSONからの値の取り出し、及び包含証明の再計算は、符号化とアルゴリズムを本書で規定します。これらは任意の手段で実装できます。

1.2.前提とする環境

コマンドの例は、OpenSSL 3.0以降と、Unix系のシェル(bash)を前提とします。例はOpenSSL 3.5で動作を確かめています。Ed25519の署名検証に`openssl pkeyutl -rawin`を用いるため、OpenSSL 1.1系では第6章の一部を実行できません。

例の中の`token.der`等のファイル名は任意です。

1.3.用意するもの

- ・ 検証の対象(タイムスタンプを付与したPDF、又は分離トークン.tstとその対象ファイル)
- ・ 当方のルート認証局証明書(2.1)
- ・ 当方の中間認証局及びルート認証局のCRL(2.2)。検証の時点で公開されている最新のものの
- ・ リポジトリの技術情報に掲げる、公開透明性ログの公開鍵とwitnessの公開鍵

1.4.運用規程4.1.4との対応

運用規程4.1.4	確認する事項	本書
第一項	存在証明が添付されていること	第5章
第二項	TSA証明書が当方のルート認証局に連なり、失効していないこと	第4章、第7章
第三項	タイムスタンプトークンの署名が有効であること	第4章
第四項	包含証明がcheckpointのroot hashと整合すること	6.2、6.3
第五項	checkpointがログの鍵で署名され、witnessの連署が有効であること	6.4～6.6
第六項	トークンの時刻がwitnessの連署が示す時刻と矛盾しないこと	6.7

第四項から第六項は、当方の関与なく検証できます。当方のサーバに問い合わせる必要があるのは、第二項の失効の確認に用いるCRLの取得だけです。

PDF閲覧ソフト(Adobe Acrobat等)の署名の表示、及びOpenSSLの`ts -verify`が確かめるのは、証明書の連なりとトークンの署名(第二項・第三項)までです。これらで有効と表示されても、トークンに記録された時刻の裏付けは確かめられていません。時刻の裏付けは、第一項及び第四項から第六項を確かめることにより得られます。これらは、TSUMUGIタイムスタンプクライアントの検証機能、又は本書の第5章及び第6章の手順により確かめられます。

2. 準備

2.1. ルート認証局証明書の取得と確認

ルート認証局証明書をリポジトリから取得し、SHA-256指紋を運用規程2.5.5に記載の値と照合します。照合はリポジトリのページではなく、運用規程の本文に対して行ってください。リポジトリの掲載内容が差し替えられた場合でも、これを検出するためです。

```
curl -sS -o root.cer https://tsumugi-ca-issuer.tsumugilabo.workers.dev/ca/root.cer
openssl x509 -inform DER -in root.cer -noout -subject -dates -fingerprint -sha256
openssl x509 -inform DER -in root.cer -out root.pem
```

指紋が一致しない場合、以降の手順を行わないでください。

2.2. CRLの取得

中間認証局とルート認証局のCRLを取得し、ルート認証局証明書と1つのファイルにまとめます。以降、このファイルを信頼の起点として用います。

```
B=https://tsumugi-ca-issuer.tsumugilabo.workers.dev
curl -sS -o intermediate.crl $B/crl/intermediate.crl
curl -sS -o root.crl $B/crl/root.crl
openssl crl -inform DER -in intermediate.crl -out intermediate.crl.pem
openssl crl -inform DER -in root.crl -out root.crl.pem
cat root.pem intermediate.crl.pem root.crl.pem > store.pem
```

中間認証局のCRLの再署名の頻度は、技術情報の第2章に掲げます(運用規程2.5.2)。失効の有無は、検証の時点で公開されている最新のCRLにより確認してください(運用規程4.1.4第二項)。以前に取得したCRLを使い回さないでください。

中間認証局証明書は、タイムスタンプトークンに同梱されています(運用規程8章)。別途取得する必要はありません。

3. タイムスタンプトークンの取り出し

3.1. 分離トークン

分離トークン(拡張子`.tst`)は、RFC 3161のTimeStampToken(CMS SignedData)をDERで符号化したものです。そのまま`token.der`として用います。対象のデータは、分離トークンと対になる元のファイルです。

同じ名前で作られる`.tst.meta`は、当方のソフトウェアが用いる補助情報です。本書の手順では用いません。

3.2. PDF

タイムスタンプを付与したPDFは、ISO 32000のドキュメントタイムスタンプ(/Type /DocTimeStamp、/SubFilter /ETSI.RFC3161)を1つ以上含みます。付与のたびに1つずつ増分更新で追加されるため、2回付与したPDFには2つ含まれます。以下、1つを「層」と呼びます。層ごとに次の手順を行い、それぞれを独立に検証します。

各層の辞書は、/ByteRange [A B C D]と/Contents <...>を持ちます。

- ・ 署名の対象は、ファイルの先頭からAバイト目を起点とするBバイトと、Cバイト目を起点とするDバイトを連結したバイト列です。そのSHA-256が、トークンのmessageImprintに一致します
- ・ 2つの範囲の間(A+BバイトからCバイトの手前まで)が/Contentsの値で、<と>に囲まれた16進文字列です。これをバイト列に戻したものがトークンです。末尾は0で埋められています。本書のOpenSSLのコマンドは、埋められたまま読み込めます

```
grep -a -o '/ByteRange *\[0-9 *\]' stamped.pdf
```

表示された各行が1つの層です。ファイルの前にあるものほど先に付与された層です。以下は、/ByteRange [0 38873 71643 1270]の層の例です。

```
F=stamped.pdf; A=0; B=38873; C=71643; D=1270
{ dd if=$F bs=1 skip=$A count=$B status=none
  dd if=$F bs=1 skip=$C count=$D status=none; } | openssl dgst -sha256
dd if=$F bs=1 skip=$((A+B+1)) count=$((C-A-B-2)) status=none \
  | xxd -r -p > token.der
```

1行目のコマンドが表示するハッシュ値(以下「対象のハッシュ値」といいます。)を、第4章で用います。

PDFに含まれる層のうち、本書の対象は当方のTSAが生成したもの、すなわちポリシーOIDが運用規程1.2.2のタイムスタンプポリシーであるもの、又はTSA証明書が当方のルート認証局に連なるものです。その他の時刻認証局による層は、当該時刻認証局の定めるところにより検証してください。

4. トークンの署名・証明書・失効の検証

4.1. トークンの内容の表示

```
openssl ts -reply -token_in -in token.der -token_out -text
```

次の項目を確認します。

表示	確認する事項
Policy OID	運用規程1.2.2のタイムスタンプポリシー(1.3.6.1.4.1.66920.1.1)であること
Hash Algorithm	sha256であること
Message data	対象のハッシュ値(3.2)又は対象ファイルのSHA-256と一致すること
Time stamp	トークンに記録された時刻(以下「genTime」といいます。)。協定世界時(GMT)で表示される

genTimeはTSA秘密鍵の保持者が記録した時刻です。その裏付けは6.7で確かめます。

4.2.署名・証明書・失効の検証

PDFの層の場合:

```
openssl ts -verify -token_in -in token.der -digest <target-hash-hex> \
  -CAfile store.pem -crl_check_all
```

分離トークンの場合:

```
openssl ts -verify -token_in -in file.tst -data file \
  -CAfile store.pem -crl_check_all
```

Verification: OKと表示されれば、次の事項が確認されたことになります。

- ・ トークンの署名が有効であること(第三項)
- ・ messageImprintが対象のデータに一致すること
- ・ TSA証明書が中間認証局を経て当方のルート認証局に連なり、TSA証明書・中間認証局証明書とも失効していないこと(第二項)

certificate revokedを理由に失敗した場合は、第7章に進みます。それ以外の理由で失敗した場合、当該トークンは有効ではありません。

4.3. TSA証明書の有効期間が過ぎている場合

TSA証明書の有効期間は2年です(運用規程6.1.3)。有効期間が過ぎた後は、4.2のコマンドに-no_check_timeを加えて実行します。この場合は証明書の有効期間を検証しないため、genTimeがTSA証明書の有効期間内にあることを、次のコマンドで別に確かめてください。

```
openssl pkcs7 -inform DER -in token.der -print_certs -text \
  | grep -E 'Subject:|Not Before|Not After'
```

失効の確認は、有効期間が過ぎた後も最新のCRLにより行います。

5.存在証明の取り出し(第一項)

5.1.属性の位置

存在証明は、トークンのSignerInfoのunsigned attributeに、OID 1.3.6.1.4.1.66920.2.1の属性として格納されています。

```
openssl asn1parse -inform DER -in token.der -i | grep -A2 '66920.2.1'
```

次のように表示されます(数値はトークンごとに異なります)。

```
1944:d=7  hl=2 l= 10 prim: OBJECT      :1.3.6.1.4.1.66920.2.1
1956:d=7  hl=4 l=2569 cons: SET
1960:d=8  hl=4 l=2565 prim: OCTET STRING [HEX DUMP]:30820A01...
```

OIDが表示されない場合、存在証明は添付されていません。当該トークンは運用規程に適合するものではなく、記録された時刻の裏付けはありません(運用規程4.1.4)。当該属性が2つ以上ある場合、又は属性の値が2つ以上ある場合も、有効ではありません。

5.2.存在証明の構造

属性の値は1つのOCTET STRINGで、その中身は次の構造のDERです。

```
TsumugiAnchorProof ::= SEQUENCE {  
    version      INTEGER (1),  
    logUrl       UTF8String,  
    retrievedAt  GeneralizedTime,  
    payloadType  OBJECT IDENTIFIER, -- 1.3.6.1.4.1.66920.2.2  
    payload      OCTET STRING      -- UTF-8 JSON  
}
```

項目	内容
version	1
logUrl	存在証明を取得した公開透明性ログのURL
retrievedAt	存在証明を取得した時刻として、生成時に記録したもの。時刻の裏付けではない
payloadType	1.3.6.1.4.1.66920.2.2(Rekor v2のエントリのJSON)。これ以外の値の場合、本書の手順では検証できない
payload	公開透明性ログの応答のJSON。ログから受け取ったバイト列を変えずに格納したもの

5.3. 取り出し

5.1で表示されたOCTET STRINGの位置(例では1960)を指定して存在証明を取り出し、続けてpayloadの位置を指定してJSONを取り出します。

```
openssl asn1parse -inform DER -in token.der -strparse 1960 -noout -out proof.der  
openssl asn1parse -inform DER -in proof.der  
openssl asn1parse -inform DER -in proof.der -strparse 82 -noout -out payload.json
```

2行目の表示で、version・logUrl・retrievedAt・payloadTypeを確認し、payload(OCTET STRING)の位置(例では82)を3行目に指定します。

6. 存在証明の検証(第四項～第六項)

6.1. payloadの構造

payloadは、Sigstore Rekor v2が返すTransparencyLogEntryのJSON表現です。本書の手順で用いる項目は次のとおりです。64ビット整数は文字列で表されます。

項目	内容
inclusionProof.logIndex	ログにおけるエントリの位置(10進の文字列)
inclusionProof.treeSize	包含証明の対象とするツリーの大きさ(10進の文字列)
inclusionProof.rootHash	ツリーのroot hash(base64)
inclusionProof.hashes	包含証明のハッシュ列(base64の配列)
inclusionProof.checkpoint.envelope	checkpointの本文(6.4)
canonicalizedBody	ログに記録されたエントリの本体(base64)

その他の項目(logId・kindVersion・integratedTime等)は、本書の手順では用いません。
Rekor v2のintegratedTimeは常に0で、時刻を表しません。

6.2. 対象データとの結びつき

canonicalizedBodyをbase64デコードすると、次の形のJSONが得られます。

```
{ "apiVersion": "0.0.2", "kind": "hashedrekord", "spec": { "hashedRekordV002": {  
  "data": { "algorithm": "SHA2_256", "digest": "<base64>" },  
  "signature": { "content": "<base64>", "verifier": { ... } } } }
```

spec.hashedRekordV002.data.digestをbase64デコードした32バイトが、トークンのmessageImprint(4.1のMessage data)と一致することを確認めます。一致しない場合、ログに記録されているのは別のデータであり、当該存在証明は有効ではありません。

参考: signatureは、TSA秘密鍵によるdigestへの署名です。verifierの公開鍵は、トークンのTSA証明書の公開鍵と一致します。

6.3. 包含証明の再計算(第四項)

RFC 6962 2.1.1のアルゴリズムにより、エントリからroot hashを再計算します。記号||は連結、SHA256はSHA-256を表します。

```
leaf = SHA256(0x00 || body)  
fn = logIndex  
sn = treeSize - 1  
r = leaf  
for each p in hashes:  
  if fn is odd or fn == sn:  
    r = SHA256(0x01 || p || r)  
    while fn != 0 and fn is even:  
      fn = fn >> 1  
      sn = sn >> 1  
  else:  
    r = SHA256(0x01 || r || p)  
  fn = fn >> 1  
  sn = sn >> 1  
require sn == 0  
require r == rootHash
```

- ・ bodyは、canonicalizedBodyをbase64デコードしたバイト列そのものです。JSONとして読み直して符号化し直したものではありません
- ・ hashesの各要素は、base64デコードした32バイトです
- ・ rは、inclusionProof.rootHashと、6.4のcheckpointの3行目のroot hashの両方と一致しなければなりません。署名されているのはcheckpointの側です

6.4. checkpointの構造

checkpointは、C2SPのsigned note形式のテキストです。改行はLF(0x0A)です。

```
log2025-1.rekor.sigstore.dev  
127963650  
Yli7MLQbPW9Snu+IS11d8P2CkQwEMjTpi5yUgleZ52M=  
  
<sig-line: log>  
<sig-line: witness 1>
```

```
<sig-line: witness 2>
...
```

- ・ 1行目がログの識別子(origin)、2行目がツリーの大きさ、3行目がroot hash(base64)です。2行目はinclusionProof.treeSizeと一致しなければなりません
- ・ 空行の後に署名行が並びます。各署名行は、U+2014(EM DASH)、空白、署名者の名前、空白、base64の署名の順に並びます
- ・ ログとwitnessが署名するのは、1行目から3行目まで(各行末のLFを含み、空行を含まない)のバイト列です。以下、これを「checkpointの本文」といいます

以下の例は、checkpointのテキストをcheckpoint.txtに保存したものとします(JSONの文字列の\nはLFに戻します)。

```
ORIGIN=$(sed -n 1p checkpoint.txt)
SIZE=$(sed -n 2p checkpoint.txt)
ROOT=$(sed -n 3p checkpoint.txt)
printf '%s\n%s\n%s\n' "$ORIGIN" "$SIZE" "$ROOT" > cp.msg
```

6.5. ログの署名の検証(第五項)

originがリポジトリの技術情報に掲げる公開透明性ログの識別子であることを確かめ、当該ログの公開鍵(Ed25519、base64のSubjectPublicKeyInfo)を用います。

署名行のうち、名前がoriginに一致するものがログの署名です。base64デコードすると68バイトで、先頭4バイトが鍵の識別子(keyhint)、残る64バイトがEd25519の署名です。

keyhintは、origin、0x0A、0x01、公開鍵32バイトをこの順に連結したもののSHA-256の先頭4バイトです。この32バイト全体が、リポジトリに掲げるログのkeyIdです。

```
echo '<log-public-key-spki-base64>' | base64 -d > log.spki.der
grep "^$(printf '\xe2\x80\x94') $ORIGIN " checkpoint.txt | awk '{print $3}' \
| base64 -d | tail -c 64 > log.sig
openssl pkeyutl -verify -pubin -keyform DER -inkey log.spki.der \
-rawin -in cp.msg -sigfile log.sig
```

Signature Verified Successfullyと表示されれば、checkpointはログの鍵で署名されています。

ログの署名は時刻を含みません。時刻を与えるのはwitnessの連署です(6.6)。

6.6. witnessの連署の検証(第五項)

名前がリポジトリの技術情報に掲げるwitnessの名称に一致する署名行について、次の手順で連署を検証します。掲げられていない名前の署名行は用いません。

witnessの公開鍵は、リポジトリに「名称+keyhint(8桁の16進)+base64」の形(vkey)で掲げています。base64の部分をデコードすると33バイトで、先頭1バイトが0x04、残る32バイトがEd25519の公開鍵です。keyhintは、名称、0x0A、0x04、公開鍵32バイトをこの順に連結したもののSHA-256の先頭4バイトです。

署名行の署名をbase64デコードすると76バイトです。

位置	長さ	内容
0	4バイト	keyhint
4	8バイト	連署の時刻。1970年1月1日0時(UTC)からの秒数、ビッグエンディアン
12	64バイト	Ed25519の署名

witnessが署名するのは、checkpointの本文の前に次の2行を置いたバイト列です(C2SP tlog-cosignature v1)。時刻は10進で、先頭に0を付けません。

```
cosignature/v1
time <seconds>
```

以下は、名称がwitness.stagemole.euの例です。

```
W=witness.stagemole.eu
echo '<vkey-base64-part>' | base64 -d | tail -c 32 > w.raw
{ printf '\x30\x2a\x30\x05\x06\x03\x2b\x65\x70\x03\x21\x00'
  cat w.raw; } > w.spki.der
grep "^$(printf '\xe2\x80\x94') $W " checkpoint.txt | awk '{print $3}' \
| base64 -d > w.bin
TS=$(16#$(xxd -p -s 4 -l 8 w.bin))
{ printf 'cosignature/v1\ntime %s\n' "$TS"; cat cp.msg; } > w.msg
tail -c 64 w.bin > w.sig
openssl pkeyutl -verify -pubin -keyform DER -inkey w.spki.der \
  -rawin -in w.msg -sigfile w.sig
date -u -d @$TS
```

2行目・3行目は、32バイトのEd25519公開鍵をSubjectPublicKeyInfoのDERに包むものです。

検証に成功した連署が1つ以上あれば、第五項を満たします。検証に成功した連署の時刻のうち最も早いものを、以下「確定時刻W」といいます。当該checkpoint、したがって当該エントリは、確定時刻Wの時点でログに存在していました。

現在、リポジトリに掲げるwitnessは、いずれも運営者が試験運用(staging)と位置付けるものです(運用規程4.8)。

6.7.時刻の照合(第六項)

genTime(4.1)と確定時刻W(6.6)の差が300秒以内であることを確かめます。両者はいずれも協定世界時です。

結果	判定
差が300秒以内	第六項を満たす。genTimeは公開透明性ログの記録により裏付けられる
genTimeがWより300秒を超えて後	矛盾がある。トークンの時刻が、ログへの登録より後を示している
genTimeがWより300秒を超えて前	矛盾がある。ログへの登録が、トークンの時刻より大きく遅れている。過去の時刻を名乗って作られたトークンはこれに当たる
検証に成功した連署が無い	時刻の裏付けを確認できない

当方のソフトウェアは、差が300秒を超える場合、及び検証できる連署が無い場合には、トークンを生成しません(運用規程4.1.3)。

retrievedAt(5.2)は、この照合に用いません。

7. TSA証明書が失効している場合

7.1. 基準日時

4.2がcertificate revokedを理由に失敗した場合、中間認証局のCRLで当該TSA証明書のエントリを確認します。

```
openssl pkcs7 -inform DER -in token.der -print_certs -text \  
| grep -A1 'Serial Number'  
openssl crl -in intermediate.crl.pem -noout -text
```

CRLの表示では、シリアル番号の先頭の0が省かれることがあります。エントリの内容により、運用規程4.6.2の基準日時Tを次のとおり定めます。

CRL Reason Code	Invalidity Date	基準日時T
Key Compromise	あり	Invalidity Dateの日時
Key Compromise	なし	TSA証明書の有効期間の開始日時
その他	—	Revocation Dateの日時

7.2. 判定

4.2のコマンドから-crl_check_allを除いて実行し、失効以外の事項(署名・証明書の連なり)を確かめます。続けて第5章・第6章の手順を行い、確定時刻Wを求めます。

条件	判定
$W < T$	有効。失効より前に正当な鍵の保持者が生成したことが、公開透明性ログの記録により確かめられる
$W \geq T$	無効
Wを確定できない	無効

Tと比べるのは確定時刻Wです。genTimeは用いません。genTimeはTSA秘密鍵を持つ者が任意に記録できるためです。

RFC 3161に従う汎用の検証ソフトウェアは、鍵の危殆化により失効したTSA証明書によるトークンを、すべて無効と表示することがあります(運用規程4.6.2)。

8. 判定のまとめ

当方のTSAが生成したトークンは、次のすべてを満たすときに有効です。

- ・ 4.2でVerification: OKとなること。失効している場合は、7.2で有効と判定されること

- ・ 存在証明が1つだけ添付されていること(第5章)
- ・ 存在証明のdigestがmessageImprintと一致すること(6.2)
- ・ 包含証明から再計算したroot hashが、inclusionProof.rootHash及びcheckpointのroot hashと一致すること(6.3)
- ・ checkpointが、リポジトリに掲げる公開透明性ログの鍵で署名されていること(6.5)
- ・ リポジトリに掲げるwitnessの連署が1つ以上検証できること(6.6)
- ・ genTimeと確定時刻Wの差が300秒以内であること(6.7)

PDFの場合は、当方のTSAによる層のそれぞれについて確かめます。

当方のソフトウェアの検証機能は、本書と同じ事項を確かめます。本書の手順と当方のソフトウェアの結果が異なる場合は、運用規程1.4の問い合わせ先へお知らせください。